实验 10-单周期处理器综合应用设计试验

一、 实验准备

本次实验为处理器综合应用设计试验，在上次搭建并仿真成功的处理基础上，将处理器运行到 FPGA 板子上进行验证。并在 Mars 上设计汇编应用程序，利用 Mars 仿真器产生二进制代码放在自己所设计的 CPU 上面运行，查看功能结果。本实验将会贯通之前所有的实验环节，打通之前所有的实验知识点；在进行本次实验前，你需要具备以下基础能力：
1.熟悉 Vivado 的仿真功能（行为仿真)；
2.单周期处理已经仿真通过；

二、 实验目的

1. 掌握单周期 CPU 的实现方法，代码实现方法；
2. 掌握测试单周期 CPU 的方法。
3. 输入输出原理；

三、 实验设备

PC 机一台，ZYNQ7020 开发板，Xilinx Vivado 开发套件。

四、 实验任务

4.1 实验要求和实验步骤

（1）按照上节课的实验，保证所设计的单周期处理器仿真通过；
（2）改造 CPU，使得 CPU 具备输入输出能力和在线调试能力；

- 寄存器堆调试接口改写：
  为了能够在上板后查看寄存器状态，需要对原有的寄存器调试接口进行改写，在 regfile.v 文件中增加 debug 接口，其中 ra_debug 通过串口输入。寄存器的结果通过 rd_debug[31:0]输出，并连接到串口显示，用于检查每个寄存器的值是否符合预期。

top.srcs > sources_1 > imports > rtl > regfile.v

```
1  module regfile(
2      input wire clk,
3      input wire we3,
4      input wire[4:0] ra1,ra2,wa3,
5      input wire[31:0] wd3,
6      output wire[31:0] rd1,rd2,
7      //add for debug
8      input wire[4:0] ra_debug, // =>connect to top moudle, physically connect to switch
9      output wire[31:0] rd_debug //=>connect to top moudle, physically connect to seg7
10     );
11     reg [31:0] rf[31:0] ;
12     always @(posedge clk) begin
13         if(we3) begin
14             rf[wa3] <= wd3;
15         end
16     end
17     assign rd1 = (ra1 != 0) ? rf[ra1] : 0;
18     assign rd2 = (ra2 != 0) ? rf[ra2] : 0;
19     //add for debug
20     assign rd_debug = (ra_debug != 0) ? rf[ra_debug] : 0;
21 endmodule
```

- 接受外部输入：
  为了能够实现外部与板子上单周期 CPU 运行程序的交互，需要引入外设(例如键盘)，这里我们使用串口模拟一个简单的外设，将串口接收到的数据保存在一个 32 位的

寄存器中，以下称为串口寄存器。通常情况下，操作系统将内存空间映射为几个不同的区域，作为系统的保留空间、程序的执行空间以及外设的保留空间(显卡的显存、键盘的缓冲区等)。这里，data memory 中 0x0-0x3FFF 为存储器空间，和之前实验 9 的使用方式完全一致，0x4000-0xFFFF_FFFF 映射为外设空间，0x4000 号地址为串口寄存器基址。通过访存指令 lw 获取该寄存器的状态，在 CPU 的实现中，引入一个 Mux，用于检查当前访问的内存区域是存储器空间还是外设空间并选通对应的数据。可以参考的一个实现方式为：

assign memrdata_actual = (memaddr_actual[14]) ? uart_reg_i : memrdata;

其中 uart_reg 为串口寄存器的值，memaddr 为送入 data_ram 的地址，dataaddr 为从 alu 得到的地址。

```
mips u_mips(
    .clk_i           (clk_i           ),
    .rst_n           (rst_n           ),
    .pc_run_en_i     (pc_run_en_i     ),
    .pc_clr_i        (pc_clr_i        ),
    .pc_o            (pc              ),
    .instr_i         (instr           ),
    .memwrite_o      (memwrite        ),
    .aluout_o        (dataaddr        ),
    .writedata_o     (writedata       ),
    .readdata_i      (readdata        ),
    .monitor_list_o  (monitor_list_o  ),
    .ra_debug_i      (ra_debug_i      ),
    .ra_debug_data_o (ra_debug_data_o )
);

inst_rom u_inst_rom (
    .clka(clk_i),    // input wire clka
    .ena(pc_run_en_i),      // input wire ena
    .addra(pc[7:2]),  // input wire [3 : 0] addra
    .douta(instr)  // output wire [31 : 0] douta
);

data_ram u_data_ram (
    .clka(~clk_i),    // input wire clka
    .ena(1'b1),      // input wire ena
    .wea(memwrite),      // input wire [0 : 0] wea
    .addra(memaddr_actual[13:0]),  // input wire [13 : 0] addra
    .dina(memwdata_actual),    // input wire [31 : 0] dina
    .douta(memrdata)  // output wire [31 : 0] douta
);
```

```
95    assign memaddr_actual = mem_debug_rden_i ? mem_debug_addr_i : memaddr;
96    assign memwdata_actual = writedata;
97    assign memrdata_actual = (memaddr_actual[14]) ? uart_reg_i : memrdata;
98
99    assign memwdata = writedata;
100   assign memaddr = dataaddr[14:0];
101   assign mem_debug_rdata_o = memrdata_actual;
102   assign readdata = memrdata_actual;
```

- 添加左移、右移指令：
  在原有的指令集基础上，加入两个 R-Type 指令 sllv (变量逻辑左移), srlv (变量逻辑右移)，为后续的上板程序提供支持。 引入新的指令只需要 ALU 解码器(alu_dec.v)产生的控制信号 alucontrol 进行拓展(编码位宽不够可以增加位宽)，并在 alu 中实现对应的逻辑左移、右移功能。编写简单的程序，验证对于这些指令集的支持，以及确保这些修改没有影响其他原有指令的正常功能。这里再次强调一遍，为了工程简洁，实验中我们不适用 shamt 域进行常数移位，可以不实现常数移位；
- 上版验证：
  - 验证寄存器 debug 功能和外设读取功能
    编写下列简单测试代码，并在 Mars 上编写汇编代码，产生二进制 coe 文件。使用串口发送数据，将数据保存在串口寄存器中，之后运行程序，通过串口查看$1 寄存器的值是否正确；

```
.text
loop:    j loop
```

  - 在 Mars 上编写汇编代码，实现简单的计算器功能:通过 lw 指令，CPU 可以读取 32 位串口状态寄存器的值，其中 31-30 位为计算器的功能位，其中功能位的定义如下：

00	01	10	11
加法	减法	乘法	除法

    其中，乘法和除法运算需要使用加减法实现(复用 mars 实验代码)
    剩下的 30 位（平均分为 15 位）分别为两个输入数；要求根据 31-30 位寄存器的值调用不同的子函数，然后将最后的计算结果放在$4 和$8 里面；并通过串口显示。假设上面的过程定义为 RIC_exe10，那么整 个程序应该是死循环保证 CPU 一直循环运行 RIC_exe10；即为：

```
While(1)
{
    整个 RIC_exe10 过程;
}
```

(3) 串口指令码

注意：此小节的“指令”是与串口监视器交互的命令，不是单周期 CPU 的指令!
注 2：完整的串口监视器操作码和结构请看附录，这里给出的指令已经足够完成实验。

串口指令码（8bit）	功能
00000000	运行
00000001	复位

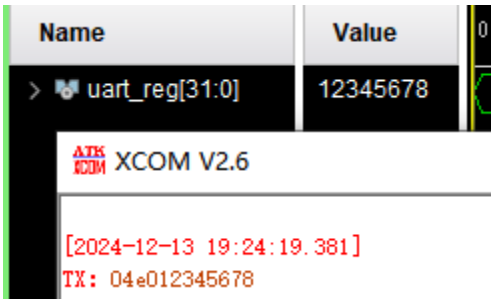
00000100	寄存器操作
00000110	单步执行
00000101	数据内存操作

- 寄存器操作

串口指令码	操作		地址码
8bit	3bit		5bit
00000100	查看某个寄存器的值	010	寄存器地址 00000-11111
	查看全部寄存器的值	110	无效，建议输入 00000

串口指令码	操作		地址码	写入数据
8bit	3bit		5bit	32bit
00000100	写入某个寄存器的值	100	寄存器地址 00000-11111	32 位数据
	写入串口寄存器的值	111	无效，建议输入 00000	32 位数据

向串口寄存器写入 32'h12345678 的样例如下所示：



- 内存操作

串口指令码	操作		地址码
8bit	1bit		14bit
00000101	查看某个地址的值	0	内存地址

读取地址 15'h054 的指令码构成如下所示：

进制	指令码
二进制	00000101 0 000000001010100
十六进制	05 0054

串口显示如下：

XCOM V2.6

```
[2024-12-15 22:39:10.399]
TX: 050054
[2024-12-15 22:39:11.405]
RX: 00 00 00 01
```

五、 实验报告要求

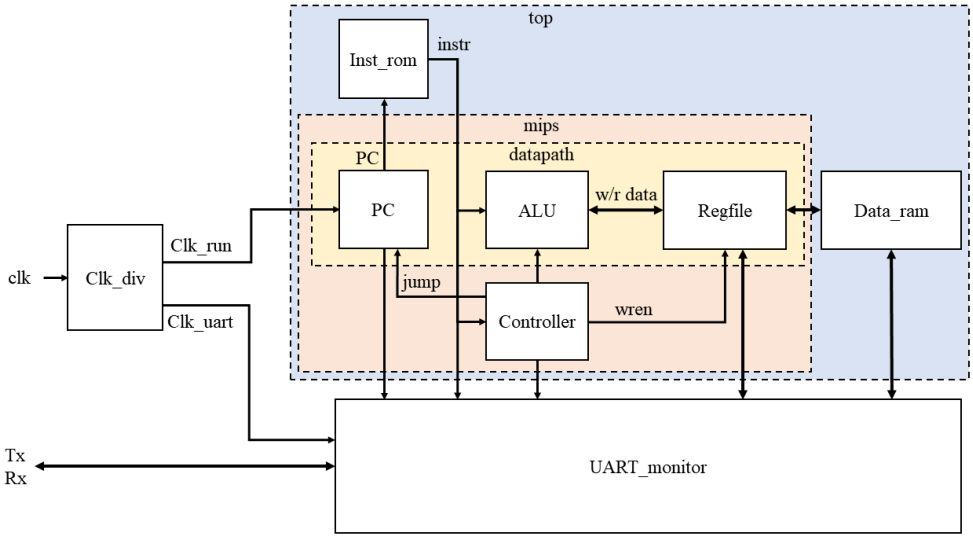
1. 按照实验要求搭建实验系统，完成寄存器堆改写和拨码开关改写，能够运行寄存器 debug 功能和外设读取功能，并上板测试；
2. 按照实验要求，在 Mars 仿真器里面实现要求的功能，并在实验报告中阐述代码设计过程，给出 Mars 仿真器运行测试结果和分析结果；
3. 将代码进行实际上板验证，验证并观察单周期 CPU 真实运行的结果，并分析结果的正确性；验收的时候，解释其运行结果的正确性；
4. 在实验报告中，详细阐述实验设计过程，代码模块结构，以及观察到实验结果以及验证原理，并分析实验结果。

附录 1 串口监视器介绍

本章节将介绍串口监视器的结构以及所有操作码，以及部分修订历史，并会介绍一些对监视器进行修改的方法。

1. 串口监视器与 CPU 交互介绍

首先给出目前单周期 CPU 的结构:



Top 中的结构仅为示例，实际实现可以自行处理。

可以看到，我们的串口监视器其实是独立于 CPU 结构以外的电路，负责将 PC 机上传输的指令解码为实际的操作行为，通过在 CPU 中预留的调试接口与 CPU 的部分电路进行交互，并将 CPU 的部分数据通过串口发送回 PC 机。

在 Lab8 中，我们已经将 Controller 的部分信号通过 monitor_list 接到了监视器上，Lab9 我们通过对 Regfile 添加一路读口接到监视器上，本次 Lab10 我们会将 Data_ram 也接到监视器上，但我们提供的代码模板中已经完成了读口的连接，不需要其它操作就可以直接读取数据 RAM 中的内容。

```
99    // uart monitor
100
101   sys_monitor #(
102       .monitor_len            (11),
103       .RX_byte                (1),
104       .TX_byte                (4)
105   )u_sys_monitor(
106       .clk_i                  (clk_uart),
107       .rst_n                  (rst_n),
108       .clk_run_i              (clk_run),
109       .rx_i                   (rx_i),
110       .tx_o                   (tx_o),
111       .monitor_list_i         (monitor_list),
112       .pc_i                   (pc),
113       .proc_cur_inst_i        (instr),
114       .proc_run_en_o          (pc_run_en),
115       .proc_reset_o           (pc_clr),
116       .pc_ov_i                (pc_ov),
117       .reg_wren_o             (),
118       .reg_addr_o             (reg_addr_dbg       ),
119       .reg_wdata_o            (reg_wdata_dbg      ),
120       .reg_rdata_i            (reg_rdata_dbg      ),
121       .uart_reg_o             (uart_reg),
122       .mem_wren_o             (),
123       .mem_rden_o             (memread_dbg),
124       .mem_addr_o             (memaddr_dbg        ),
125       .mem_wdata_o            (),
126       .mem_rdata_i            (memrdata_dbg       )
127   );
```

PC

Reg

RAM

在 CPU 中对应的模块添加接口，并连接到串口监视器上，即可完成操作。

2. 串口操作码

目前串口监视器已经实现了以下功能：

a) 启动 PC
b) 清零 PC
c) 打印当前指令
d) 设置断点
e) 寄存器操作
f) 数据内存操作
g) 单步执行

同时串口监视器也拥有一个可使用的 32bit 寄存器，被映射到内存 0x4000 地址上。
部分指令会有输出，在对应指令码中会有说明。
下面是所有指令的具体操作码与示例：

串口指令码（8bit）	功能
00000000	启动 PC
00000001	清零 PC
00000010	打印当前指令
00000011	设置断点
00000100	寄存器操作
00000110	单步执行
00000101	数据内存操作

- 启动 PC

  串口输入 00 后，会将 pc_run_en 置高，同时监视器内会将 monitor_list 上的数据送入串口发送端，直到 pc_run_en 置低。
  输出的内容是 24bit 的 monitor_list 与 8bit 的 PC，PC 输出低 8 位，monitor_list 不足 24 位会补零，但不能超过 24 位。

```
[2024-12-15 20:22:21.654]
TX: 00
[2024-12-15 20:22:22.696]
RX: 2D 00 00 00 14 40 00 04 14 40 00 08 14 40 00 0C 0C 20 00 10 0C 00 00 14 0C 40 00
18 00 C0 00 1C 2D E0 00 20 21 C0 00 24 35 40 00 2C 2D 40 00 30 0C C0 00 34 50 40 00
38 94 40 00 3C 23 40 00 40 14 40 00 44 50 40 00 48
```

- 清零 PC

  串口输入 01 后，会将 pc_clr 置高一拍，此时 PC 应该被置 0。

- 打印当前指令

  串口输入 02，会输出当前 PC 所对应的指令。

```
[2024-12-15 20:15:42.954]
TX: 02
[2024-12-15 20:15:43.963]
RX: 20 02 00 05
```

- 设置断点

串口指令码	PC 指针
8bit	32bit
00000100	需要停下的指令对应的 PC 指针

```
[2024-12-15 20:22:52.012]
TX: 0300000008
[2024-12-15 20:22:55.546]
TX: 00
[2024-12-15 20:22:56.554]
RX: 2D 00 00 00 14 40 00 04 14 40 00 08
[2024-12-15 20:22:57.997]
TX: 00
[2024-12-15 20:22:59.031]
RX: 14 40 00 0C 0C 20 00 10 0C 00 00 14 0C 40 00 18 00 C0 00 1C 2D E0 00 20 21 C0 00
24 35 40 00 2C 2D 40 00 30 0C C0 00 34 50 40 00 38 94 40 00 3C 23 40 00 40 14 40 00
44 50 40 00 48
[2024-12-15 20:23:11.427]
TX: 03ffffffff
[2024-12-15 20:23:14.058]
TX: 01
[2024-12-15 20:23:16.889]
TX: 00
[2024-12-15 20:23:17.913]
RX: 2D 00 00 00 14 40 00 04 14 40 00 08 14 40 00 0C 0C 20 00 10 0C 00 00 14 0C 40 00
18 00 C0 00 1C 2D E0 00 20 21 C0 00 24 35 40 00 2C 2D 40 00 30 0C C0 00 34 50 40 00
38 94 40 00 3C 23 40 00 40 14 40 00 44 50 40 00 48
```

通过指令发送需要断点的 PC 地址后，若当前 PC 与断点 PC 一致，pc_run_en 会被拉低，再次发送 00 后会继续执行指令。断点地址被设为 FFFF_FFFF 表示取消断点。

- 寄存器操作

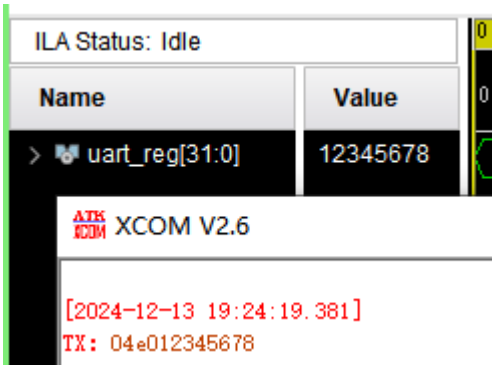
串口指令码	操作		地址码
8bit	3bit		5bit
00000100	查看某个寄存器的值	010	寄存器地址 00000-11111
	查看全部寄存器的值	110	无效，建议输入 00000

串口指令码	操作		地址码	写入数据
8bit	3bit		5bit	32bit
00000100	写入某个寄存器的值	100	寄存器地址 00000-11111	32 位数据
	写入串口寄存器的值	111	无效，建议输入 00000	32 位数据

读取$7 寄存器的值示例如下：

```
[2024-12-15 20:27:56.933]
TX: 0447
[2024-12-15 20:27:57.940]
RX: FF FF FF FB
```

向串口寄存器写入 32'h12345678 的样例如下所示：



这一寄存器被暴露在 sys_monitor 的 uart_reg_o 端口上。需要注意写入寄存器是没有回复的。

- 单步执行

  会将 pc_run_en 单独拉高一拍，同时将 monitor_list 发送一次。

```
[2024-12-15 20:29:28.156]
TX: 06
[2024-12-15 20:29:29.170]
RX: 2D 00 00 00
[2024-12-15 20:29:29.645]
TX: 06
[2024-12-15 20:29:30.659]
RX: 14 40 00 04
```

- 数据内存操作

串口指令码	操作		地址码	数据
8bit	1bit		15bit	32bit
00000101	写入某个地址的值	1	内存地址	待写入数据
	读取某个地址的值	0	内存地址	不需要填写

读取地址 15'h054 的样例如下所示：

```
[2024-12-15 20:32:02.732]
TX: 050054
[2024-12-15 20:32:03.734]
RX: 00 00 00 01
```

对地址 15'h028 写入后再读出示例如下(写入口需要自己连接)：

```
[2024-12-15 20:36:41.117]
TX: 050028
[2024-12-15 20:36:42.134]
RX: 00 00 00 00
[2024-12-15 20:36:48.193]
TX: 05802887654321
[2024-12-15 20:36:52.496]
TX: 050028
[2024-12-15 20:36:53.509]
RX: 87 65 43 21
```

这一指令可以访问到串口寄存器，但不能写入。

```
[2024-12-15 20:32:49.167]
TX: 04e012345678
[2024-12-15 20:32:55.171]
TX: 054000
[2024-12-15 20:32:56.180]
RX: 12 34 56 78
```

3. 串口监视器的输出

```
always @(posedge clk_run_i or negedge rst_n) begin
    if(~rst_n) begin
        r2u_fifo_wdata <= 32'h0;
    end else begin
        casex({proc_inst_print, mem_rdata_tx, reg_rdata_tx})
            3'b000:
                r2u_fifo_wdata <= {
                                    monitor_list_i,
                                    {(TX_byte*8-monitor_len-8){1'b0}},
                                    pc_i[7:0],
                                    proc_cur_inst_i};
            3'b001:
                r2u_fifo_wdata <= {reg_rdata_i, proc_cur_inst_i};
            3'b01x:
                r2u_fifo_wdata <= {mem_rdata_i, proc_cur_inst_i};
            3'b1xx:
                r2u_fifo_wdata <= {proc_cur_inst_i, proc_cur_inst_i};
        endcase
    end
end
```

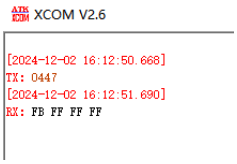
如图，这段代码位于 sys_monitor 中，用于控制送往 r2u_data_fifo 的数据，可以看到在默认情况下，串口调试器的输出由 monitor_list 与 pc 组成。同时，这一段代码也决定了 monitor_list 的最大长度为 24，否则需要将 pc 去除。低 32 位为指令码，在该模块中，这一 64bit 数据在经过 FIFO 转换时钟域后，会取高 32 位作为 TX 的输入。

```
// uart interface
assign tx_data = r2u_fifo_rdata[63:32];
assign rx_ready = ~u2r_fifo_full;
```

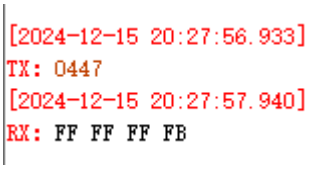
如果在这里改为 r2u_fifo_rdata[31:0]，则输出会变成指令码。这一操作可以用于查看执行的所有指令码。

在 Lab8 中，我们曾经提到数据存在大小端转换问题，在串口调试助手中会把低 8 位放在前面，Lab9 中依然存在这个问题；但在 Lab10 的模板中，我们已经解决了这个问题，现在串口调试助手中显示的数据与 Verilog 代码中定义的方向一致了。

Lab9 中的显示效果(实际寄存器值为 FFFFFFFB):



Lab10 中的效果:



修改点在 uart_top.v 的第 221 行:

```
正在比较文件 uart_top.v 和 UART_TOP_OLD.V
***** uart_top.v
        for (i = 0; i < 8; i = i + 1) begin: tx_data_sel
            assign inner_tx_data[i] = tx_data_reg[(tx_bytelen-tx_cnt-1)*8+i];
        end
***** UART_TOP_OLD.V
        for (i = 0; i < 8; i = i + 1) begin: tx_data_sel
            assign inner_tx_data[i] = tx_data_reg[tx_cnt*8+i];
        end
*****
```

4. 为 Regfile 和 datamem 添加写端口

在 Regfile.v 中已经预留了端口与操作逻辑

```
52
53      assign wr_addr = we_dbg_i ? wa_dbg_i : wa3_i;
54      assign wr_data = we_dbg_i ? wd_dbg_i : wd3_i;
55
```

sys_monitor 上已经留出了 reg 的写端口，连接上即可使用。如果不打算使用，在例化时需要将 we_dbg_i 与 wa_dbg_i 置 0。

在 top.v 中，已经例化了 data_ram 并连接了对应端口。

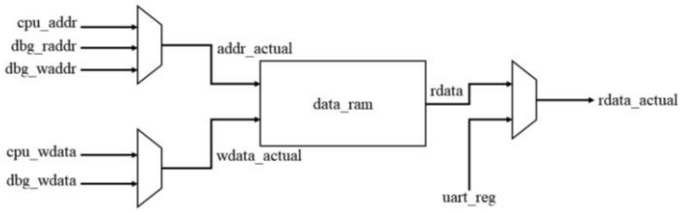
```
data_ram u_data_ram (
    .clka(~clk_i),    // input wire clka
    .ena(1'b1),      // input wire ena
    .wea(memwrite),      // input wire [0 : 0] wea
    .addra(memaddr_actual[13:0]),  // input wire [13 : 0] addra
    .dina(memwdata_actual),    // input wire [31 : 0] dina
    .douta(memrdata)  // output wire [31 : 0] douta
);

assign memaddr_actual = mem_debug_rden_i ? mem_debug_addr_i : memaddr;
assign memwdata_actual = writedata;
assign memrdata_actual = (memaddr_actual[14]) ? uart_reg_i : memrdata;

assign memwdata = writedata;
assign memaddr = dataaddr[14:0];
assign mem_debug_rdata_o = memrdata_actual;
```

若需要新增一路写端口，可以参考 regfile 中的写法，通过 mem_dbg_wren 来选通地址

与写数据，对应的电路图如下所示：



cpu_addr 为来自 cpu 的访存地址，dbg_addr 为来自串口监视器的访存地址，通过复用器选择对应端口送至 data_ram 即可。当然不写也可以完成 Lab10。

5. 常见问题 QA

- 输入 00 后没有回复

  由于 pc_run_en 拉高时会立刻启动串口发送流程，因此串口没有接收到数据说明 pc_run_en 没有拉高，有可能是 FIFO 的配置存在问题，也有可能是分频时钟质量较差。

  解决方法：检查 FIFO 配置，使用 IP Catalog 中的 Clock Wizard 进行分频。

- 输入 00 后输出了很多乱七八糟的数据

  检查每 32 位数据中的最后 8 位 PC 值，观察是否发生变化，如果没有变化说明 PC 没有运行；如果发生变化可能是 pc_ov 没有拉高，没有在执行完成时将 pc_run_en 拉低。